

Showcase: Parsing Classical Chinese Texts

The Jupyter Notebook is a showcase on how to use Yasuoka BERT models to parse Classical Chinese texts.

Lei Lei and Haobo Zhang

Shanghai International Studies University

```
import glob
import os
import re

import pandas as pd
from transformers import pipeline
```

Use Yasuoka BERT models to parse Classical Chinese texts

First, load the Yasuoka BERT models.

```
nlp = pipeline(
    "universal-dependencies",
    # will download the model at the first use
    "KoichiYasuoka/roberta-classical-chinese-large-ud-goeswith",
    # Or download it manually, and use it from a local path
    # "./roberta-classical-chinese-large-ud-goeswith",
    trust_remote_code=True,
    aggregation_strategy="simple",
)
# device = 0) # 0: cpu, 1: gpu
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 0%| | 0/391 [00:00<?, ?it/s]

Then, define a function to parse a string.

The function will return a dataframe containing columns as follows:

['token_id', 'token', 'token_pos', 'token_feats', 'head_id', 'dependency_relation']

```
# define a function to parse a string
# return a dataframe
```

```
def parse_text(mytxt: str):

    myresults = nlp(mytxt)

    myresults2 = myresults.split("\n")

    # delete the first line
    del myresults2[0]

    # delete the last two lines
    del myresults2[-1]
    del myresults2[-1]

    mydf = pd.DataFrame([line.split("\t") for line in myresults2]).iloc[
        :, [0, 2, 3, 5, 6, 7]
    ]
    mydf.columns = [
        "token_id",
        "token",
        "token_pos",
        "token_feats",
        "head_id",
        "dependency_relation",
    ]

    return mydf
```

See the following example.

Note that when you a Yasuoka model to parse a Classical Chinese string, you may need to first split the string into sentences and remove all punctuation marks.

Then parse each sentence separately.

The following is an example of the parsing of a single sentence.

```
my_sentence = "太祖姓楊名行密字化源廬州合肥人也"

parse_text(my_sentence)
```

	token_id	token	token_pos	token_feats	head_id	dependency_relation
0	1	太	VERB	Degree=Pos VerbForm=Part	2	amod
1	2	祖	NOUN	_	3	nmod

	token_id	token	token_pos	token_feats	head_id	dependency_relation
2	3	姓	NOUN	—	8	nsubj:outer
3	4	楊	PROPN	NameType=Sur	12	nsubj
4	5	名	NOUN	—	4	flat
5	6	行密	PROPN	NameType=Giv	4	flat
6	7	字	NOUN	—	8	nsubj
7	8	化源	PROPN	NameType=Giv	12	nsubj
8	9	廬	PROPN	Case=Loc Name- Type=Geo	10	nmod
9	10	州	NOUN	Case=Loc	11	nmod
10	11	合肥	PROPN	Case=Loc Name- Type=Geo	12	nmod
11	12	人	NOUN	—	0	root
12	13	也	PART	—	12	discourse:sp

Batch preprocessing: split raw Classical Chinese texts into a File – Paragraph – Sentence structured CSV

Before handing a long passage to the Yasuoka model as a single block, do two things first:

1. **Remove punctuation and split into sentences.** The model is trained at clause/sentence granularity, so the raw text should first be split on punctuation marks (which are then discarded) before each resulting sentence is sent to the model on its own.
2. **Add three marker columns.** Once split, a bare sentence can no longer be traced back to where it came from, so every row needs:
 - `file_id` / `file_name`: which file (which history book) the sentence comes from
 - `para_id`: which paragraph within that file the sentence belongs to (paragraphs are split on blank lines/line breaks in the raw text, so a source with several paragraphs will yield several `para_id` values)
 - `sent_id`: the sentence's position within that paragraph

With these three markers in place, once the model's token-level output is concatenated into a larger dataframe, it stays fully possible to recover exactly which character belongs to which sentence, which sentence belongs to which paragraph, and which paragraph belongs to which text.

```
# Punctuation set used for sentence splitting (these characters are discarded
once the text is split)
```

```

SPLIT_PUNCT = ",. !?; ::\\"'"'「」『』《》()()""

SPLIT_PATTERN = re.compile(f"[{re.escape(SPLIT_PUNCT)}]")

def segment_text(raw_text: str):
    """
    Split a block of raw text into a list of (para_id, sent_id, sentence) tuples.
    Paragraphs are first split on blank lines/line breaks; each paragraph is
    then
    split into individual clauses/sentences on punctuation, with the punctuation
    itself discarded.
    """
    rows = []
    paragraphs = [p.strip() for p in re.split(r"\n+", raw_text) if p.strip()]
    for para_id, para in enumerate(paragraphs, start=1):
        pieces = SPLIT_PATTERN.split(para)
        sent_id = 0
        for piece in pieces:
            piece = piece.strip()
            if piece:
                sent_id += 1
                rows.append((para_id, sent_id, piece))
    return rows

def write_csv(file_id: str, file_name: str, raw_text: str, out_dir: str):
    """
    Segment raw text and write it out as a CSV that follows the
    File - Paragraph - Sentence three-level marker schema.
    Columns: file_id, file_name, para_id, sent_id, sentence
    """
    rows = segment_text(raw_text)
    os.makedirs(out_dir, exist_ok=True)
    out_path = os.path.join(out_dir, file_name)
    out_df = pd.DataFrame(rows, columns=["para_id", "sent_id", "sentence"])
    out_df.insert(0, "file_name", file_name)
    out_df.insert(0, "file_id", file_id)
    out_df.to_csv(out_path, index=False, encoding="utf-8-sig")
    return out_path, len(rows)

```

In the following examples, we use the opening passages of five classic annals/biographies, i.e., the *Records of the Grand Historian*, *Book of Han*, *Book of the Later Han*, *Records of the Three Kingdoms*, and *Book of Jin*, to generate five .csv files that are already sentence-split, punctuation-free, and carry the three marker columns.

You can also drop your own raw text into the TEXTS dictionary below and rerun the cell below to get a .csv file in the same format.

TEXTS = {

"01 史记-汉-司马迁.csv": "黄帝者，少典之子，姓公孙，名曰轩辕。生而神灵，弱而能言，幼而徇齐，长而敦敏，成而聪明。\\n 轩辕之时，神农氏世衰。诸侯相侵伐，暴虐百姓，而神农氏弗能征。于是轩辕乃习用干戈，以征不享，诸侯咸来宾从。而蚩尤最为暴，莫能伐。炎帝欲侵陵诸侯，诸侯咸归轩辕。轩辕乃修德振兵，治五气，艺五种，抚万民，度四方，教熊罴貔貅虎，以与炎帝战于阪泉之野。三战然后得其志。蚩尤作乱，不用帝命。于是黄帝乃徵师诸侯，与蚩尤战于涿鹿之野，遂禽杀蚩尤。而诸侯咸尊轩辕为天子，代神农氏，是为黄帝。天下有不顺者，黄帝从而征之，平者去之，披山通道，未尝宁居。"，

"02 汉书-汉-班固.csv": "高祖，沛丰邑中阳里人也，姓刘氏。母媪尝息大泽之陂，梦与神遇。是时雷电晦冥，父太公往视，则见交龙于上。已而有娠，遂产高祖。\\n 高祖为人，隆准而龙颜，美须髯，左股有七十二黑子。宽仁爱人，意豁如也。常有大度，不事家人生产作业。及壮，试吏，为泗上亭长，廷中吏无所不狎侮。好酒及色。常从王媪、武负貰酒，时饮醉卧，武负、王媪见其上常有怪。高祖每酤留饮，酒讎数倍。及见怪，岁竟，此两家常折券弃责。"，

"03 后汉书-南朝宋-范曄.csv": "世祖光武皇帝讳秀，字文叔，南阳蔡阳人，高祖九世之孙也，出自景帝生长沙定王发。发生舂陵节侯买，买生郁林太守外，外生钜鹿都尉回，回生南顿令钦，钦生光武。光武年九岁而孤，养于叔父良。身長七尺三寸，美须眉，大口，隆准，日角。性勤于稼穡，而兄伯升好侠养士，常非笑光武事田业，比之高祖兄仲。王莽天凤中，乃之长安，受尚书，略通大义。\\n 莽末，天下连岁灾蝗，寇盗蜂起。地皇三年，南阳荒饥，诸家宾客多为小盗。光武避吏新野，因卖谷于宛。宛人李通等以图讖说光武云：“刘氏复起，李氏为辅。”光武初不敢当，然独念兄伯升素结轻客，必举大事，且王莽败亡已兆，天下方乱，遂与定谋，于是乃市兵弩。十月，与李通从弟轶等起于宛，时年二十八。"，

"04 三国志-晋-陈寿.csv": "太祖武皇帝，沛国谯人也，姓曹，讳操，字孟德，汉相国参之后。\\n 太祖一名吉利，小字阿瞞。王沈魏书曰：其先出于黄帝。当高阳世，陆终之子曰安，是为曹姓。周武王克殷，存先世之后，封曹侠于邾。春秋之世，与于盟会，逮至战国，为楚所灭。子孙分流，或家于沛。汉高祖之起，曹参以功封平阳侯，世袭爵土，绝而复绍，至今适嗣国于容城。桓帝世，曹腾为中常侍大长秋，封费亭侯。司马彪续汉书曰：腾父节，字元伟，素以仁厚称。邻人有亡豕者，与节豕相类，诣门认之，节不与争；后所亡豕自还其家，豕主人大慙，送所认豕，并辞谢节，节笑而受之。由是乡党贵叹焉。长子伯兴，次子仲兴，次子叔兴。腾季李兴，少除黄门从官。永宁元年，邓太后诏黄门令选中黄门从官年少温谨者，配皇太子书，腾应其选。太子特亲爱腾，饮食赏赐与衆有异。顺帝即位，为小黄门，迁至中常侍大长秋。在省闼三十餘年，历事四帝，未嘗有过。好进达贤能，终无所毁伤。其所称荐，若陈留虞放、边韶、南阳延固、张温、弘农张奂、颍川堂溪典等，皆致位公卿，而不伐其善。蜀郡太守因计吏修敬于腾，益州刺史种暠于函谷关搜得其笈，上太守，并奏腾内臣外交，所不当为，请免官治罪。帝曰：“笈自外来，腾书不出，非其罪也。”乃实鬲奏。腾不以介意，常称叹鬲，以为鬲得事上之节。鬲后为司徒，语人曰：“今日为公，乃曹常侍恩也。”腾之行事，皆此类也。桓帝即位，以腾先帝旧臣，忠孝彰著，封费亭侯，加位特进。太和三年，追尊腾曰高皇帝。养子嵩嗣，官至太尉，莫能审其生出本末。续汉书曰：嵩字巨高。质性敦慎，所在忠孝。为司隸校尉，灵帝擢拜大司农、大鸿臚，代崔烈为太尉。黄初元年，追尊嵩曰太皇帝。吴人作曹瞞传及郭颁世语并云：嵩，夏侯氏之子，夏侯敦之叔父。太祖于敦为从父兄弟。嵩生太祖。"，

"05 晋书-唐-房玄龄.csv": "宣皇帝讳懿，字仲达，河内温县孝敬里人，姓司马氏。其先出自帝高阳之子重黎，为夏官祝融，历唐、虞、夏、商，世序其职。及周，以夏官为司马。其后程柏休父，周宣王时，以世官克平徐方，锡以官族，因而为氏。楚汉间，司马仰为赵将，与诸侯伐秦。秦亡，立为殷王，都河内。汉以其地为郡，子孙遂家焉。自仰八世，生征西将军钧，字叔平。钧生豫章太守量，字公度。量生颍川太守俊，字元异。俊生京兆尹防，字建公。帝即防之第二子也。少有奇节，聪明多大略，博学洽闻，伏膺儒教。汉末大乱，常慨然有忧天下心。南阳太守同郡杨俊名知人，见帝，未弱冠，以为非常之器。尚书清河崔琰与帝兄朗善，亦谓朗曰：“君弟聪亮明允，刚断英特，非子所及也。”\\n 汉建安六年，郡举上计掾。魏武帝为司空，闻而辟之。帝知汉运方微，不欲屈节曹氏，辞以风痺，不能起居。魏武使人夜往密刺之，帝坚卧不动。及魏武为丞相，又辟为文学掾，敕行者曰：“若复盘桓，便收之。”帝惧而就职。于是，使与太子游处，迁黄门侍郎，转议郎、丞相东曹属，寻转主簿。从讨张鲁，言于魏武曰：“刘备以诈力虏刘璋，蜀人未附而远争江陵，此机不可失也。今若曜威汉中，益州震动，进兵临之，势必瓦解。因此之势，易为功力。圣人不能违时，亦不失时矣。”魏武曰：“人苦无足，既得陇右，复欲得蜀！”言竟不从。既而从讨孙权，破之。军还，权遣使乞降，上表称臣，陈

```

说天命。魏武帝曰：“此儿欲踞吾着炉炭上邪！”答曰：“汉运垂终，殿下十分天下而有其九，以服事之。权之
称臣，天人之意也。虞、夏、殷、周不以谦让者，畏天知命也。””
}

```

```

data_dir = "test_data"
for file_id_num, (fname, text) in enumerate(TEXTS.items(), start=1):
    fid = fname[:2]
    path, n = write_csv(fid, fname, text, data_dir)
    print(f"{fname}: {n} sentences -> {path}")

```

```

01 史记-汉-司马迁.csv: 42 sentences -> test_data\01 史记-汉-司马迁.csv
02 汉书-汉-班固.csv: 33 sentences -> test_data\02 汉书-汉-班固.csv
03 后汉书-南朝宋-范晔.csv: 46 sentences -> test_data\03 后汉书-南朝宋-范晔.csv
04 三国志-晋-陈寿.csv: 118 sentences -> test_data\04 三国志-晋-陈寿.csv
05 晋书-唐-房玄龄.csv: 110 sentences -> test_data\05 晋书-唐-房玄龄.csv

```

Preview what one of the CSV files looks like (five columns: file_id / file_name / para_id / sent_id / sentence):

```

pd.read_csv(os.path.join("test_data", "01 史记-汉-司马迁.csv")).head(10)

```

	file_id	file_name	para_id	sent_id	sentence
0	1	01 史记-汉-司马迁.csv	1	1	黄帝者
1	1	01 史记-汉-司马迁.csv	1	2	少典之子
2	1	01 史记-汉-司马迁.csv	1	3	姓公孙
3	1	01 史记-汉-司马迁.csv	1	4	名曰轩辕
4	1	01 史记-汉-司马迁.csv	1	5	生而神灵
5	1	01 史记-汉-司马迁.csv	1	6	弱而能言
6	1	01 史记-汉-司马迁.csv	1	7	幼而徇齐
7	1	01 史记-汉-司马迁.csv	1	8	长而敦敏
8	1	01 史记-汉-司马迁.csv	1	9	成而聪明
9	1	01 史记-汉-司马迁.csv	2	1	轩辕之时

Parse sentence by sentence, and re-attach the File / Paragraph / Sentence markers to the token-level output

Read every CSV in the test_data/ folder that follows the five-column schema, call parse_text() on each sentence in turn, and then attach that sentence's file_id, file_name, para_id, and sent_id to every token row produced for it.

Each input file produces its own output file (as many output files as input files), named as the original file stem plus `_parsed`, and saved into the `test_output/` folder.

Individual sentences that fail to parse do not stop the whole run. They are collected into `test_output/failed_sentences.csv` for later review.

```
os.makedirs("test_output", exist_ok=True)

failed_sentences = []

csv_files = sorted(glob.glob(os.path.join("test_data", "*.csv")))

for csv_path in csv_files:
    sent_df = pd.read_csv(csv_path)
    file_parsed = []
    for _, row in sent_df.iterrows():
        sentence = str(row["sentence"])
        try:
            parsed = parse_text(sentence)
        except Exception as e:
            failed_sentences.append(
                {
                    "file_name": row["file_name"],
                    "para_id": row["para_id"],
                    "sent_id": row["sent_id"],
                    "sentence": sentence,
                    "error": str(e),
                }
            )
            continue

        # Re-attach the three-level markers (and the original sentence text) to
        # every token row
        parsed.insert(0, "file_id", row["file_id"])
        parsed.insert(1, "file_name", row["file_name"])
        parsed.insert(2, "para_id", row["para_id"])
        parsed.insert(3, "sent_id", row["sent_id"])
        parsed.insert(4, "sentence", sentence)
        file_parsed.append(parsed)

    if file_parsed:
        file_parsed_df = pd.concat(file_parsed, ignore_index=True)
    else:
        file_parsed_df = pd.DataFrame()

    stem = os.path.splitext(os.path.basename(csv_path))[0]
    out_path = os.path.join("test_output", f"{stem}_parsed.csv")
    file_parsed_df.to_csv(out_path, index=False, encoding="utf-8-sig")
```

```

    print(
        f"{stem}: {len(sent_df)} sentences -> {len(file_parsed_df)} tokens -> {out_path}"
    )

if failed_sentences:
    pd.DataFrame(failed_sentences).to_csv(
        os.path.join("test_output", "failed_sentences.csv"),
        index=False,
        encoding="utf-8-sig",
    )
    print(
        f"{len(failed_sentences)} sentence(s) failed to parse; written to test_output/failed_sentences.csv"
    )

```

```

01 史记-汉-司马迁: 42 sentences -> 199 tokens -> test_output\01 史记-汉-司马迁_parsed.csv
02 汉书-汉-班固: 33 sentences -> 151 tokens -> test_output\02 汉书-汉-班固_parsed.csv
03 后汉书-南朝宋-范晔: 46 sentences -> 227 tokens -> test_output\03 后汉书-南朝宋-范晔_parsed.csv
04 三国志-晋-陈寿: 118 sentences -> 565 tokens -> test_output\04 三国志-晋-陈寿_parsed.csv
05 晋书-唐-房玄龄: 110 sentences -> 496 tokens -> test_output\05 晋书-唐-房玄龄_parsed.csv

```